

El procesador

Departamento de Enseñaría de Computadores

- 1 Tipo de procesadores
 - Introducción
- 2 Repertorio de instrucciones del MIPS
- 3 Repertorio de instrucciones
 - Unidad Central de Proceso
 - Diseño del repertorio de instrucciones
- 4 Paralelismo a nivel de instrucciones
 - Dependencias y paralelismo a nivel de instrucción
 - Riesgos en la ejecución

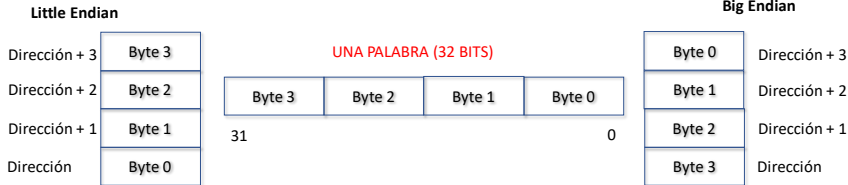
Funcionamiento de un computador

- Un computador funciona ejecutando continuamente instrucciones que operan sobre diferentes datos
- Instrucciones y datos se almacenan en memoria y se codifican empleando un número específico de bits
- El computador está diseñado para transferir bloques de información de un determinado tamaño
 - A cada uno de estos bloques se le denomina **palabra**
 - El tamaño, en número de bits, de cada palabra se denomina **ancho de palabra**
- Todos los elementos del computador (registros, buses, memorias, periféricos, ALUs) están diseñados para transferir, almacenar o procesar palabras.

Representación de los bytes en una palabra

El *endian* especifica el orden de los bytes si un elemento de datos tiene varios bytes:

- *Little endian* significa que el byte de orden inferior del número se almacena en la memoria en la dirección más baja y el byte de orden superior en la dirección más alta.
- *Big endian* significa que el byte de orden superior del número se almacena en la memoria en la dirección más baja, y el byte de orden inferior en la dirección más alta.



Arquitectura del MIPS R2000

Registros

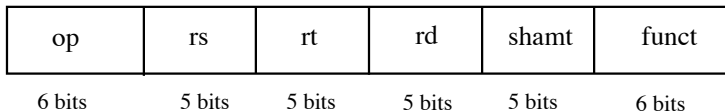
Nombre	Número	Uso	Preservado en llamada
\$zero	0	Valor constante 0	n.a.
\$at	1	Reservado para el ensamblador	n.a.
\$v0-\$v1	2-3	Valores para resultados de subrutinas	no
\$a0-\$a3	4-7	Paso de parámetros	sí
\$t0-\$t7	8-15	Registros temporales	no
\$s0-\$s7	16-23	Registros que deben preservarse	sí
\$t8-\$t9	24-25	Registros temporales	no
\$k0-\$k1	26-27	Reservado para núcleo de SO	n.a.
\$gp	28	Puntero global	sí
\$sp	29	Puntero de pila	sí
\$fp	30	Puntero de bloque de activación	sí
\$ra	31	Dirección de retorno	sí

Tipos de instrucciones MIPS

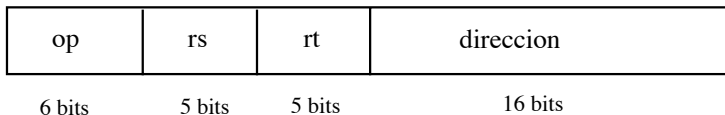
- Tipos de instrucciones soportados:
 - Transferencia de datos
 - Aritméticas y lógicas
 - De control de flujo:
 - Salto condicional
 - Bifurcación
- Características:
 - Longitud de todas las instrucciones fija de 32 bits.
 - Los operandos de las operaciones aritméticas son siempre registros
 - El acceso a memoria se hace a través de operaciones de carga/almacenamiento (transferencia de datos).

Formatos de las instrucciones MIPS

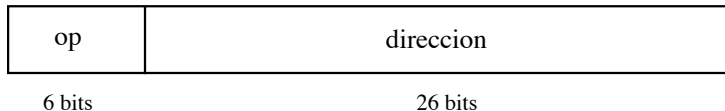
Tipo - R



Tipo - I



Tipo - J



Formatos de las instrucciones MIPS

Instrucciones aritmético-lógicas:

- add, sub, or, and, ...
- Formato tipo R
- Direcccionamiento de registro

add \$7,\$3,\$6

0	3	6	7	0	32
---	---	---	---	---	----

000000	00011	00110	00111	00000	100000
--------	-------	-------	-------	-------	--------

31 25 20 15 10 5 0

sub \$7,\$3,\$6

0	3	6	7	0	34
---	---	---	---	---	----

000000	00011	00110	00111	00000	100010
--------	-------	-------	-------	-------	--------

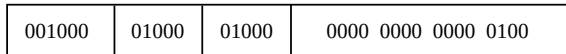
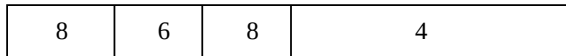
31 25 20 15 10 5 0

Formatos de las instrucciones MIPS

Operaciones aritméticas con operandos inmediatos:

- addi, andi, ori, ...
- Formato tipo I
- Direccionamiento de registro + direccionamiento inmediato

addi \$8,\$6,4



31 25 20 15 0

Formatos de las instrucciones MIPS

Operaciones de transferencia:

- lw (*load word*) y sw (*store word*)
- Tipo I
- Direccionamiento con desplazamiento

lw \$8,1200(\$15)

35	15	8	1200
----	----	---	------

100011	01111	01000	0000 0100 1011 0000
31	25	20	15
			0

sw \$8,1200(\$15)

43	15	8	1200
----	----	---	------

101011	01111	01000	0000 0100 1011 0000
31	25	20	15
			0

Formatos de las instrucciones MIPS

Operaciones de transferencia con operandos inmediatos:

- lui (*load upper immediate*)
- Tipo I
- Direccionamiento inmediato

Version en lenguaje maquina: lui \$8,255

001111	00000	01000	0000 0000 1111 1111
--------	-------	-------	---------------------

Contenido del registro \$8
tras la ejecucion:

0000 0000 1111 1111	0000 0000 0000 0000
---------------------	---------------------

(\$rt=inm;16)

Formatos de las instrucciones MIPS

Instrucciones de salto condicional:

- beq (*branch if equal*) y bne (*branch if not equal*)
- Tipo I
- Direccionamiento con desplazamiento relativo
- beq \$rs,\$rt,despl salta a la instrucción que se encuentra en la dirección de memoria $(PC+4)+4*\text{despl}$ si $\$rs == \rt .
- El ensamblador permite el uso de etiquetas para indicar la dirección de salto

4	19	20	L1
---	----	----	----

000100	10011	10100	0001 1000 0011 1101
--------	-------	-------	---------------------

5	19	20	L1
---	----	----	----

000101	10011	10100	0001 1000 0011 1101
--------	-------	-------	---------------------

IC.GIA: tema 3

Formatos de las instrucciones MIPS

Instrucciones de bifurcación:

- *j* (*jump*): Tipo J, direccionamiento pseudodirecto
 - $PC \leftarrow (PC + 4)_{31-28}IR_{25-0}00$
 - El ensamblador permite el uso de etiquetas
- *jr* (*jump register*): Tipo R, direccionamiento indirecto con registro.

j L1

2	L1
---	----

jr \$8

0	8	0	0	0	8
---	---	---	---	---	---

Formatos de las instrucciones MIPS

- Para hacer comparaciones del tipo *menor que* tenemos la instrucción lógica **slt**
- `slt $rd,$rs,$rt: $rd=($rs<$rt)`
- `slti $rd,$rs,inm: $rd=($rs<inm)`

`slt $8,$19,$20`

0	19	20	8	0	42
---	----	----	---	---	----

`slti $8,$19,10`

10	19	8	10
----	----	---	----

- `slt`: Tipo R, direccionamiento de registro
- `slti`: Tipo I, direccionamiento de registro + inmediato

Las instrucciones máquina

- El funcionamiento de la CPU está determinado por las instrucciones que ejecuta
- El conjunto de instrucciones distintas que puede ejecutar se denomina **repertorio de instrucciones**
- El repertorio de instrucciones condiciona la implementación
- Cada instrucción debe contener toda la información necesaria para su ejecución

Las instrucciones máquina

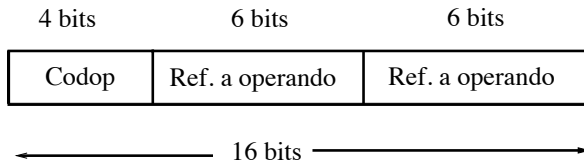
- Los elementos constitutivos de una instrucción máquina son:
 - Código de operación
 - Referencia a operandos fuente: uno o más
 - Referencia al operando resultado: si produce alguno
 - Referencia a la siguiente instrucción: en la mayoría de los casos es implícito y se refiere a la siguiente instrucción de la lista. En caso contrario, se suministrará la dirección de memoria

Las instrucciones máquina

- Los elementos constitutivos de una instrucción máquina son:
 - Código de operación
 - Referencia a operandos fuente: uno o más
 - Referencia al operando resultado: si produce alguno
 - Referencia a la siguiente instrucción: en la mayoría de los casos es implícito y se refiere a la siguiente instrucción de la lista. En caso contrario, se suministrará la dirección de memoria
- Los operandos fuente y resultado estarán en alguna de las siguientes localizaciones:
 - Memoria: deberá indicarse la dirección
 - Registro de la CPU: identificados por un número único
 - Dispositivo de E/S: módulo y dispositivo E/S o dirección de memoria en caso de E/S asignada en memoria

Las instrucciones máquina

- La instrucción se divide en campos correspondientes a los elementos constitutivos de la misma (código de operación, operandos, etc.)
- La división en campos y bits se denomina **formato de instrucción**.



- La mayoría de los repertorios de instrucciones emplean más de un formato

Las instrucciones máquina

- Los aspectos más importantes en el diseño del repertorio de instrucciones son:
 - Repertorio de operaciones: Cuántas y qué operaciones considerar, y cuán complejas deben ser.
 - Tipos de datos: Los distintos tipos de datos con los que se efectúan operaciones.
 - Formatos de instrucciones: Longitud de la instrucción (en bits), tamaño de los distintos campos, etc.
 - Registros: Número de registros de la CPU que pueden ser referenciados por instrucciones y su uso.
 - Direccionamiento: El modo o modos de direccionamiento mediante los cuales puede especificarse la dirección de un operando.

Modelo de ejecución

- El modelo de ejecución especifica el dispositivo (memoria, registro, etc.) que almacena los operandos de las instrucciones.
- Una misma arquitectura suele dar soporte a varios modelos de ejecución.
- Los modelos de ejecución posibles son:
 - Modelo de pila: POP y PUSH.
 - Modelo registro-registro: el modo más rápido. Se necesita transferir los datos.
 - Modelo registro-memoria: un operando en registro, otro en memoria. El resultado se almacena en un registro.
 - Modelo memoria-memoria: No se necesita transferir los datos. La ejecución requiere un alto número de ciclos.
- Los modelos soportados dependen fuertemente de la arquitectura.
 - Por ejemplo: una ALU exclusivamente conectada con registros quedará limitada al modo registro-registro.

Operaciones del repertorio de instrucciones

- Se pueden categorizar las operaciones de un repertorio de instrucciones en los siguientes tipos:
 - Transferencias de datos: *load* y *store*
 - Aritméticas
 - Lógicas
 - De E/S
 - De control de flujo:
 - Instrucciones de bifurcación o salto
 - Instrucciones de salto condicional
 - Instrucciones de llamada a subrutina
 - De control del sistema

Unidad Central de Proceso

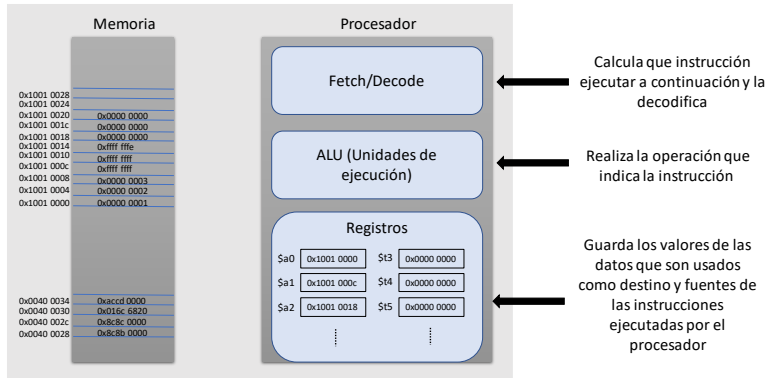
Introducción

- La Unidad Central de Proceso (CPU) es la encargada de la ejecución de las instrucciones.
- La CPU se divide en dos bloques fundamentales:
 - Camino de datos: realiza las operaciones requeridas por las instrucciones máquina. Contiene:
 - Unidad aritmético-lógica.
 - Banco de registros.
 - Registros especiales: IR, PC, etc.
 - Buses internos: caminos de conexión entre los elementos de la CPU.
 - Unidad de control: gestiona el secuenciamiento de las operaciones ejecutadas en el camino de datos.

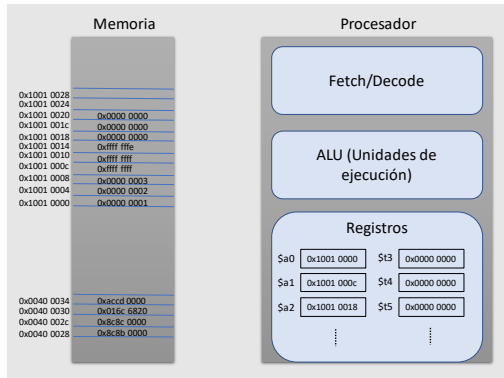
Ejecución de una instrucción

- La ejecución de una instrucción es un proceso dividido en cinco pasos:
 - **Captar la instrucción:** el procesador debe leer la instrucción de memoria.
 - **Interpretar la instrucción:** la instrucción debe decodificarse para determinar qué acción es necesaria.
 - **Captar datos:** la ejecución puede exigir leer datos de la memoria o de un módulo de E/S.
 - **Procesar datos:** la ejecución de una instrucción puede exigir llevar a cabo alguna operación aritmética o lógica.
 - **Escribir datos:** los resultados de la ejecución pueden tener que ser escritos en la memoria o en un módulo de E/S.

Ejecución de instrucciones



Ejecución de instrucciones

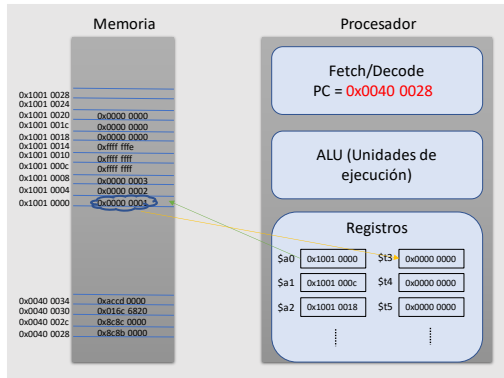


```

.
.
.
0x0040 0028 lw $t3, 0($a0)
0x0040 002c lw $t4, 0($a1)
0x0040 0030 add $t5, $t3, $t4
0x0040 0034 sw $t5, 0($a2)
.
.
.

```

Ejecución de instrucciones



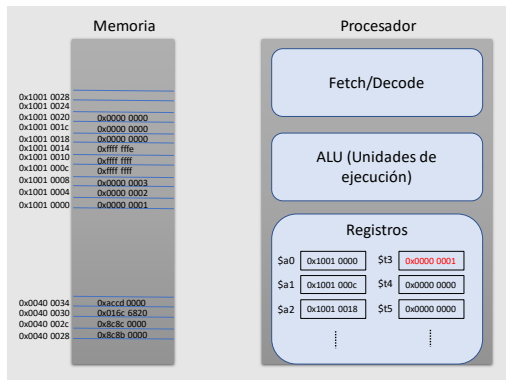
```

.
.
.
0x0040 0028  lw $t3, 0($a0)
0x0040 002c  lw $t4, 0($a1)
0x0040 0030  add $t5, $t3, $t4
0x0040 0034  sw $t5, 0($a2)
.
.
.

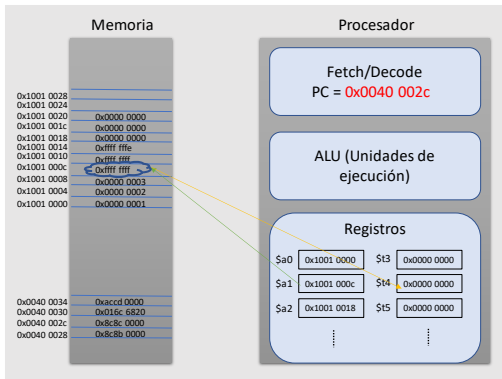
```

LOAD (CARGA)

Ejecución de instrucciones



Ejecución de instrucciones



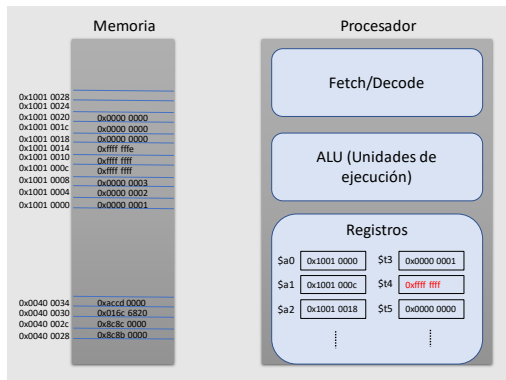
```

.
.
.
0x0040 0028  lw $t4, 0($a0)
0x0040 002c  lw $t4, 0($a1)
0x0040 0030  add $t5, $t3, $t4
0x0040 0034  sw $t5, 0($a2)
.
.
.

```

LOAD (CARGA)

Ejecución de instrucciones

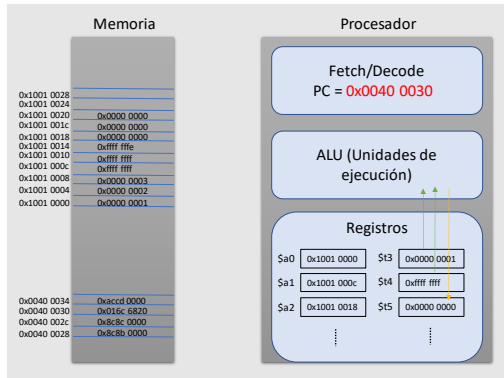


```

.
.
.
0x0040 0028 lw $t3, 0($a0)
0x0040 002c lw $t4, 0($a1)
0x0040 0030 add $t5, $t3, $t4
0x0040 0034 sw $t5, 0($a2)
.
.
.

```

Ejecución de instrucciones



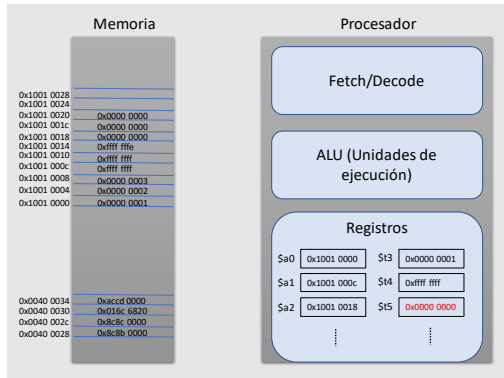
```

.
.
.
0x0040 0028  lw $t3, 0($a0)
0x0040 002c  lw $t4, 0($a1)
0x0040 0030  add $t5, $t3, $t4
0x0040 0034  sw $t5, 0($a2)
.
.
.

```

**OPERACIÓN
ARITMÉTICA**

Ejecución de instrucciones

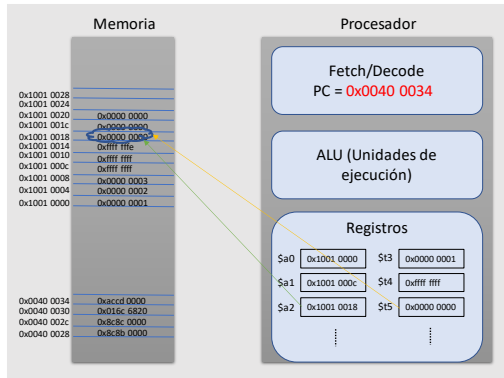


```

.
.
.
0x0040 0028 lw $t3, 0($a0)
0x0040 002c lw $t4, 0($a1)
0x0040 0030 add $t5, $t3, $t4
0x0040 0034 sw $t5, 0($a2)
.
.
.

```

Ejecución de instrucciones



```

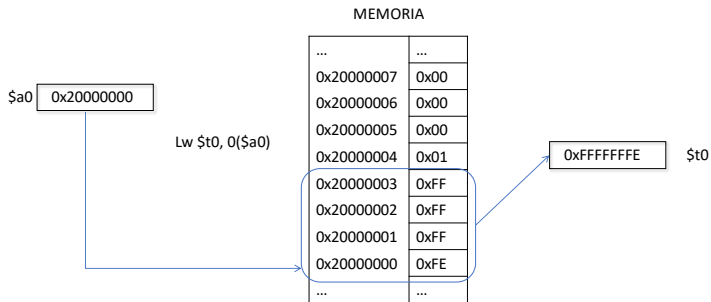
.
.
.
0x0040 0028 lw $t3, 0($a0)
0x0040 002c lw $t4, 0($a1)
0x0040 0030 add $t5, $t3, $t4
0x0040 0034 sw $t5, 0($a2)
.
.
.

```

STORE(ALMACENAMIENTO)

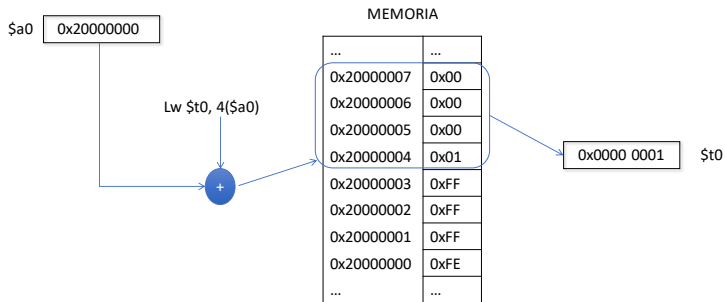
Modos de direccionamiento

Modos de direccionamiento a memoria



Modos de direccionamiento

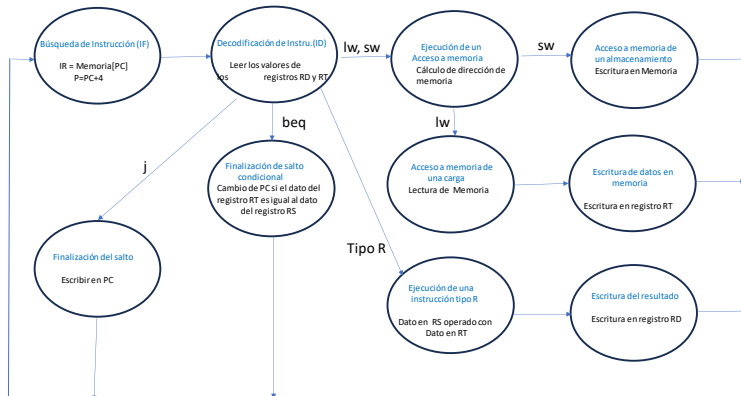
Modos de direccionamiento a memoria



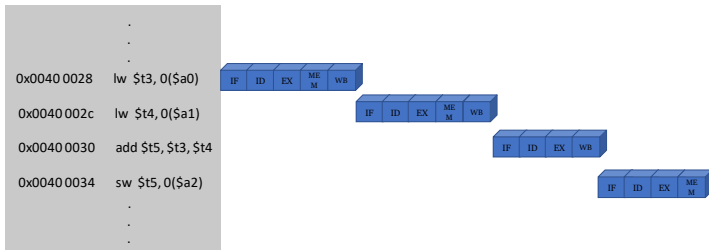
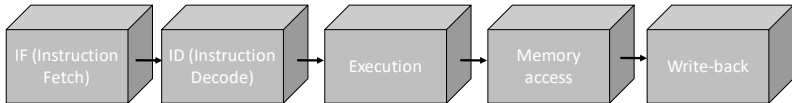
Diseño del repertorio de instrucciones

- Dos filosofías a la hora de diseñar un repertorio de instrucciones:
 - *CISC (Complex Instruction Set Computer)*
 - Juego de instrucciones muy rico
 - Modos de direccionamiento potentes y complejos
 - Menos instrucciones de ejecución compleja
 - *RISC (Reduced Instruction Set Computer)*
 - Número relativamente pequeño de instrucciones disponibles
 - Sencillas y con pocos modos de direccionamiento
 - Más instrucciones de ejecución muy eficiente
- Estudiaremos el repertorio del MIPS (*Microprocessor without Interlocked Pipeline Stages*):
 - Ejemplo típico de diseño RISC.
 - Usado por NEC, Nintendo, SGI, Sony, ...

CPU Multiciclo *(Etapas en la ejecución de instrucciones)*



MIPS (*Microprocessor without Interlocked Pipeline Stages*)



Temporización

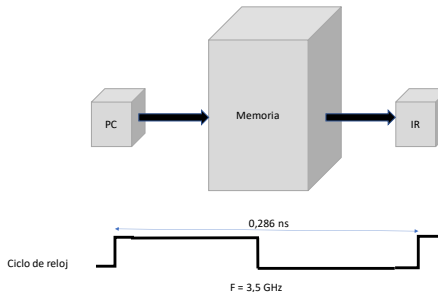
Ciclo de reloj

- El ciclo de reloj debería ser al menos tan largo como la más larga de las operaciones del camino de datos
- Ejemplo:
 - lectura de memoria: 2.5 ns
 - escritura en memoria: 3 ns
 - operación ALU: 2ns
 - acceso registros (lectura y escritura): 1ns
- Ciclo de reloj $\geq 3\text{ns}$
- Instrucción aritmética $\geq 12\text{ ns } (4 \times 3)$

Temporización

Ciclo de reloj

El procesador completa una transferencia interna y cálculos de datos en un ciclo de reloj. Es decir, cada paso debe ser realizado en un ciclo de reloj, excepto la etapa de *EX* que requiere uno o más ciclo de reloj, dependiendo de la velocidad de la unidad funcional usada.



Mejoras del rendimiento en los procesadores actuales

- Hay dos posibilidades para incrementar el rendimiento:
 - Mejorar la tecnología de los circuitos integrados, lo cual reduciría el tiempo necesario para ejecutar cada una de las etapas y permitiría reducir el ciclo de reloj.
 - Mejorar el diseño del procesador:
 - **Procesador Segmentado:** Trabaja con varias instrucciones a la vez, cada una en una etapa diferente. Se pretende conseguir la ejecución de una instrucción por ciclo (límite máximo).
 - **Procesador Superscalar:** Implementa paralelismo a nivel de instrucción. Ejecuta más de 1 instrucción por ciclo de reloj gracias a la replicación de unidades funcionales.
- Todas las CPUs de propósito general actuales son segmentadas y superescalares.

Segmentación de cauce

- La segmentación del cauce:
 - Persigue aumentar las prestaciones
 - Divide la ejecución de una instrucción en etapas
 - Permite solapar la ejecución de distintas instrucciones en diferentes etapas
 - Permite comenzar una instrucción en cada ciclo de reloj (caso ideal)
- Etapas de una instrucción MIPS en el camino de datos:
 - **IF**: búsqueda de instrucciones
 - **ID**: decodificación de instrucciones y lectura del banco de registros
 - **EX**: ejecución o cálculo de direcciones
 - **MEM**: acceso a memoria para buscar un dato
 - **WB**: escritura de resultados

Segmentación de cauce

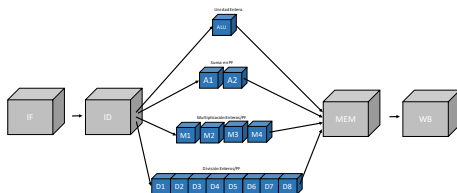
Comparación entre la implementación multiciclo y la segmentada:

Instrucciones	1	2	3	4	5	6	7	8	9	10	11	12
lw \$1,16(\$2)	IF	ID	EX	MEM	WB							
addi \$2,\$2,4						IF	ID	EX	MEM			
beq \$3,\$0,loop										IF	ID	EX

Instrucciones	1	2	3	4	5	6	7	8	9	10	11	12
lw \$1,16(\$2)	IF	ID	EX	MEM	WB							
addi \$2,\$2,4		IF	ID	EX	MEM	WB						
beq \$3,\$0,loop			IF	ID	EX	MEM	WB					

Operaciones multiciclo

- Algunas operaciones tardan más de un ciclo de reloj en la etapa de ejecución
- Las instrucciones en punto flotante tienen la misma segmentación que las enteras, pero con las siguientes modificaciones:
 - La etapa EX tiene diferente latencia dependiendo de la instrucción
 - Existen diversas unidades funcionales para cada tipo de operación



Definición

El paralelismo a nivel de instrucción (*Instruction Level Parallelism-ILP*) es la capacidad de procesar instrucciones en paralelo. Viene determinado por el número de instrucciones que pueden solaparse en las etapas de un procesador.

Dos instrucciones son dependientes si se deben ejecutar en orden:

- ① Indican la posibilidad de un riesgo (una instrucción no se puede ejecutar en el ciclo de reloj que le correspondería)
- ② Determinan el orden en el cual se deben calcular los resultados.
- ③ Establecen la cantidad máxima de paralelismo que se puede obtener.

Las dependencias se pueden clasificar en:

- verdaderas
- de nombre
- de control

Dependencias verdaderas

Una instrucción depende de los resultados de una instrucción anterior, de forma que ambas no podrían ejecutarse de forma solapada.

Una instrucción j tiene una dependencia verdadera con la instrucción i si ocurre cualquiera de las situaciones siguientes:

- La instrucción i produce un resultado que puede ser usado por la instrucción j .

```
i: lwc1    $f0, 0($r1)
j: add.s   $f2, $f0, $f4
```

- La instrucción j tiene una dependencia de datos con la instrucción k , y la instrucción k tiene una dependencia de datos con la instrucción i .

```
i: lwc1    $f0, 0($r1)
k: mul.d   $f4, $f0, $f10
j: add.d    $f2, $f6, $f4
```

Dependencias de nombre

Una dependencia de nombre se produce, cuando dos instrucciones usan el mismo registro o la misma posición de memoria, pero no hay flujo de datos entre las instrucciones asociadas a ese nombre.

Hay dos tipos de dependencias de nombre entre una instrucción i que preceda a una instrucción j en el orden del programa:

- 1 Se produce una **antidependencia** entre la instrucción i y la j cuando la instrucción j escribe un registro o una posición de memoria que lee la instrucción i .

i: mul.d \$f4, \$f0, \$f10

j: add.d \$f0, \$f6, \$f8

- 2 Una **dependencia de salida** se produce cuando la instrucción i y la instrucción j escriben en el mismo registro o en la misma posición de memoria.

i: mul.d \$f4, \$f0, \$f10

j: add.d \$f4, \$f6, \$f8

Dependencia de nombre

Una dependencia de nombre no es una dependencia verdadera, las instrucciones involucradas en una dependencia de nombre pueden ejecutarse simultáneamente o ser reordenadas.

Si se cambia el nombre desaparece el conflicto **renombrado de registros**. Puede hacerse estáticamente por el compilador o dinámicamente por el hardware.

Dependencias de control

Una dependencia de control determina el orden de una instrucción con respecto a una instrucción de salto.

Tipo de restricciones:

- 1 Una instrucción con dependencia de control con un salto, no se puede mover de forma que su ejecución ya no esté controlada por el salto.

```

    if  P1{
        S1;
    }
  
```

S1 es dependiente de **P1**, es decir, sólo se ejecuta si se cumple **P1**.

- 2 Una instrucción que no tiene una dependencia de control, no se puede poner de forma que su ejecución sea controlada por el salto.

```

    S2
    if  P1{
        S1;
    }
  
```

Riesgos en la ejecución

- El CPI ideal de un procesador segmentado es uno
- Sin embargo, hay situaciones, denominadas **riesgos** (**hazards**), que impiden que se ejecute la siguiente instrucción del flujo de instrucciones durante su ciclo de reloj.

Tipos de riesgos

- **Estructurales:** el hardware no puede soportar la combinación de instrucciones que se quieren ejecutar en el mismo ciclo
- **De control:** surgen del problema de determinar la instrucción correcta que se tiene que ejecutar después de un salto
- **De datos:** surgen de la existencia de dependencias entre las instrucciones

Técnicas básicas de tratamiento de los riesgos de datos

- **Software:** el compilador ordena la secuencia de instrucciones de acuerdo a las siguientes reglas:
 - genera una secuencia de instrucciones independientes, haciendo que los riesgos desaparezcan
 - inserta instrucciones **nop** cuando no existen instrucciones independientes

```
add $1, $2, $3  
sub $7, $1, $2  
add $8, $6, $2  
and $9, $10, $2
```

```
add $1, $2, $3  
add $8, $6, $2  
and $9, $10, $2  
sub $7, $1, $2
```

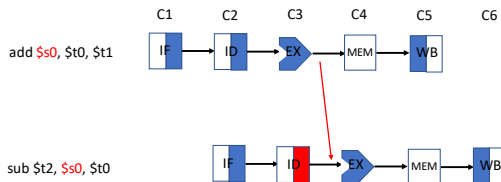
Técnicas básicas de tratamiento de los riesgos de datos

Hardware:

- **Anticipación:** el hardware adelanta el dato que hace falta una vez éste está calculado, sin necesidad de esperar a la escritura en el registro
- **Bloqueo:** el hardware para la ejecución de las instrucciones en el cauce para esperar a que el dato necesario esté disponible

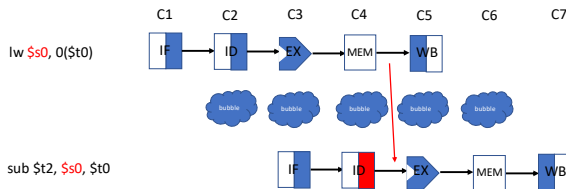
Anticipación

Anticipación: el hardware adelanta el dato que hace falta una vez éste está calculado, sin necesidad de esperar a la escritura en el registro



Bloqueo

Bloqueo: el hardware para la ejecución de las instrucciones en el cauce para esperar a que el dato necesario esté disponible



Ejemplos

